

Git Command Cheatsheet

A printable beginner-friendly reference for solo projects, branching, and connecting to GitHub or other hosted Git services.

Setup: install identity and defaults

<code>git --version</code>	Check that Git is installed.	Run first
<code>git config --global user.name "Your Name"</code>	Set the author name used in commits.	One time
<code>git config --global user.email "you@example.com"</code>	Set the author email used in commits.	One time
<code>git config --global init.defaultBranch main</code>	Use main as the default branch name for new repos.	Optional
<code>git config --global --list</code>	Show your global Git settings.	Check setup

Start a project

<code>git init</code>	Create a new Git repository in the current folder.	Local first
<code>git clone <repo-url></code>	Copy an existing hosted repository to your computer.	Remote first
<code>git status</code>	Show changed files and the current branch.	Use often
<code>git add <file></code>	Stage one file for the next commit.	Prepare
<code>git add .</code>	Stage all current changes in this folder.	Prepare
<code>git commit -m "Message"</code>	Save staged changes with a short message.	Snapshot

Everyday solo workflow

<code>git status</code>	See what changed.	Check
<code>git add .</code>	Stage the changes you want to keep.	Prepare
<code>git commit -m "Describe change"</code>	Commit a local snapshot.	Save
<code>git pull</code>	Bring down remote changes before pushing.	Sync
<code>git push</code>	Upload local commits to the remote.	Publish

View history and changes

<code>git log --oneline --graph --decorate</code>	Compact visual history.	History
<code>git diff</code>	Show unstaged changes.	Before add
<code>git diff --staged</code>	Show staged changes.	Before commit
<code>git show <commit></code>	Show what changed in a commit.	Inspect

Git Command Cheatsheet

Branching, hosted repos, undo commands, and quick safety rules.

Branches for a basic single-user setup

<code>git branch</code>	List local branches.	View
<code>git switch -c feature-name</code>	Create and switch to a new branch.	Start work
<code>git switch main</code>	Switch back to main.	Return
<code>git merge feature-name</code>	Merge a finished branch into the current branch.	Usually on main
<code>git branch -d feature-name</code>	Delete a branch after it has been merged.	Cleanup
<code>git push -u origin feature-name</code>	Push a new branch and track it on the remote.	Publish branch

Connect to GitHub, GitLab, Bitbucket, or other hosted Git repos

<code>git remote -v</code>	Show connected remotes.	Check
<code>git remote add origin <repo-url></code>	Connect a local repo to a hosted repo.	One time
<code>git branch -M main</code>	Rename the current branch to main.	Common setup
<code>git push -u origin main</code>	Push main and set upstream tracking.	First push
<code>git pull origin main</code>	Pull from the hosted main branch.	Manual sync
<code>git remote set-url origin <new-url></code>	Change the remote URL.	Move repo

Common remote URL patterns

<code>https://github.com/user/repo.git</code>	HTTPS remote URL format.	Works widely
<code>git@github.com:user/repo.git</code>	SSH remote URL format.	Needs SSH key
<code>https://gitlab.com/user/repo.git</code>	GitLab HTTPS format.	Example
<code>git@bitbucket.org:user/repo.git</code>	Bitbucket SSH format.	Example

Undo and fix safely

<code>git restore <file></code>	Discard unstaged changes in one file.	Careful
<code>git restore --staged <file></code>	Unstage a file but keep its edits.	Safe
<code>git commit --amend</code>	Edit the last commit or add staged changes to it.	Before push
<code>git revert <commit></code>	Create a new commit that undoes an older commit.	Safe after push
<code>git reset --hard <commit></code>	Move history back and discard changes.	Danger

Ignore files

<code>touch .gitignore</code>	Create an ignore file on macOS/Linux.	Optional
<code>echo node_modules/ >> .gitignore</code>	Ignore dependency folders.	Example
<code>echo .env >> .gitignore</code>	Ignore secrets and local config.	Important
<code>git rm --cached <file></code>	Stop tracking a file but keep it locally.	After ignore

Simple rule: status -> add -> commit -> pull -> push. Use branches for experiments, features, or risky changes. Do not commit passwords, API keys, .env files, or private credentials.